

Aula 13 - 19/05/2026

Introdução ao Java Web e Javalin

Sumário

- Apresentação do ecossistema: Jakarta EE, Servidores e a produtividade dos Frameworks.
- Preparação do ambiente web: Criação do primeiro projeto com Javalin.
- Construção do primeiro get "Hello World" e de get JSON para: data/hora do servidor
 - TAREFA: Implementar um '/greeting', que responda um texto plain com a data/hora e a mensagem "bom dia", "boa tarde", "boa noite" ou "vá dormir" em função do horário
- Construção de get JSON para: números aleatórios (parâmetro n)
- Construção de get JSON para: tabuada de n (parâmetro)
- Construção de get JSON para: soma de n1 e n2 (parâmetros).
 - TAREFA: Implementar 'loteria' retornando um JSON com a data/hora e um array com 6 números aleatórios não repetidos para tentar a sorte na loteria.

O Ecossistema Java Web

Jakarta EE (antigo Java EE):

- Não é um software, mas sim um conjunto de **especificações** (regras/contratos) para construir aplicações empresariais.
- Exemplos de especificações: **Servlets** (base de toda a web Java), **JPA** (persistência), **CDI** (injeção de dependência).

Servidores de Aplicação (Containers):

- São os softwares que **implementam** as especificações do Jakarta EE.
- Eles fornecem o "container" onde nossa aplicação roda.
- **Exemplos Clássicos:**
 - **Tomcat:** O mais popular. É um *Servlet Container* (implementa principalmente a especificação de Servlets).
 - i. <https://tomcat.apache.org/>
 - **WildFly (antigo JBoss) e GlassFish:** Servidores de aplicação completos (implementam todas, ou quase todas, as especificações).
 - i. <https://www.wildfly.org/>

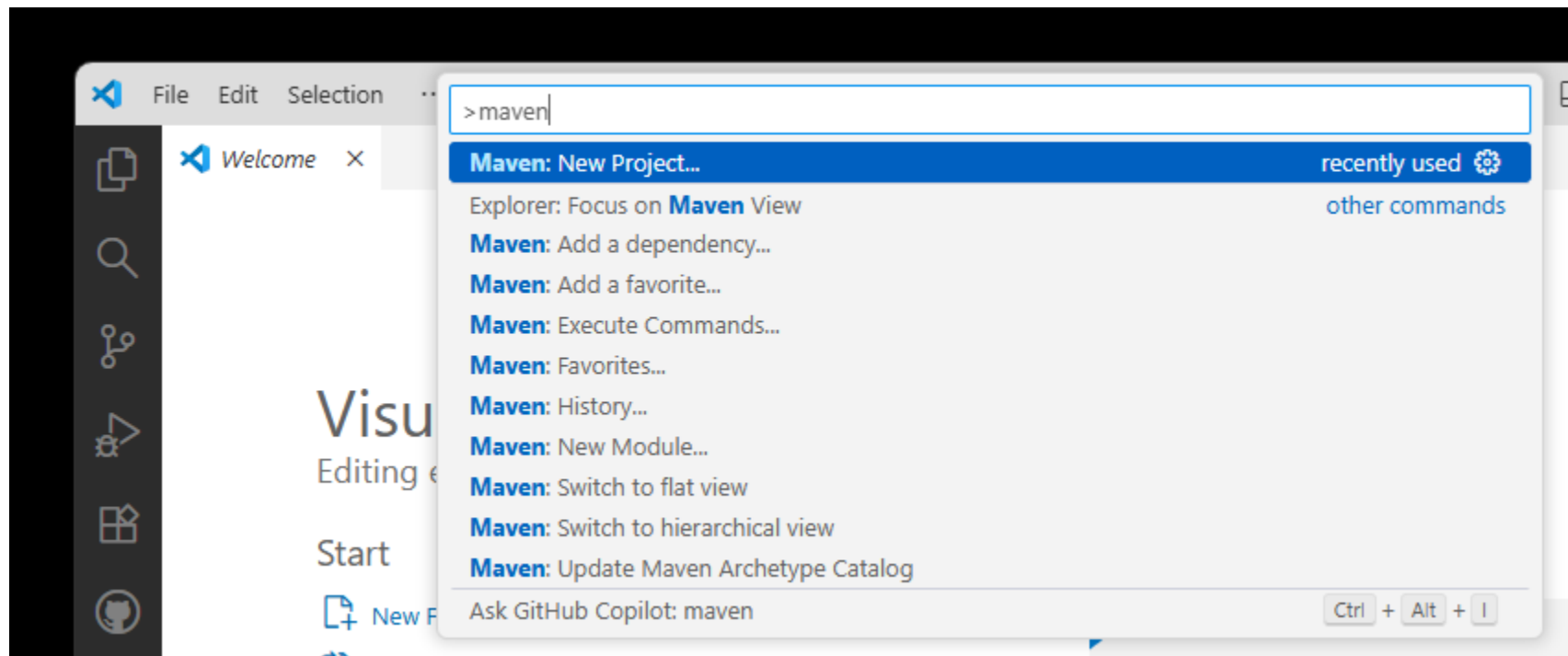


O Ecossistema Java Web

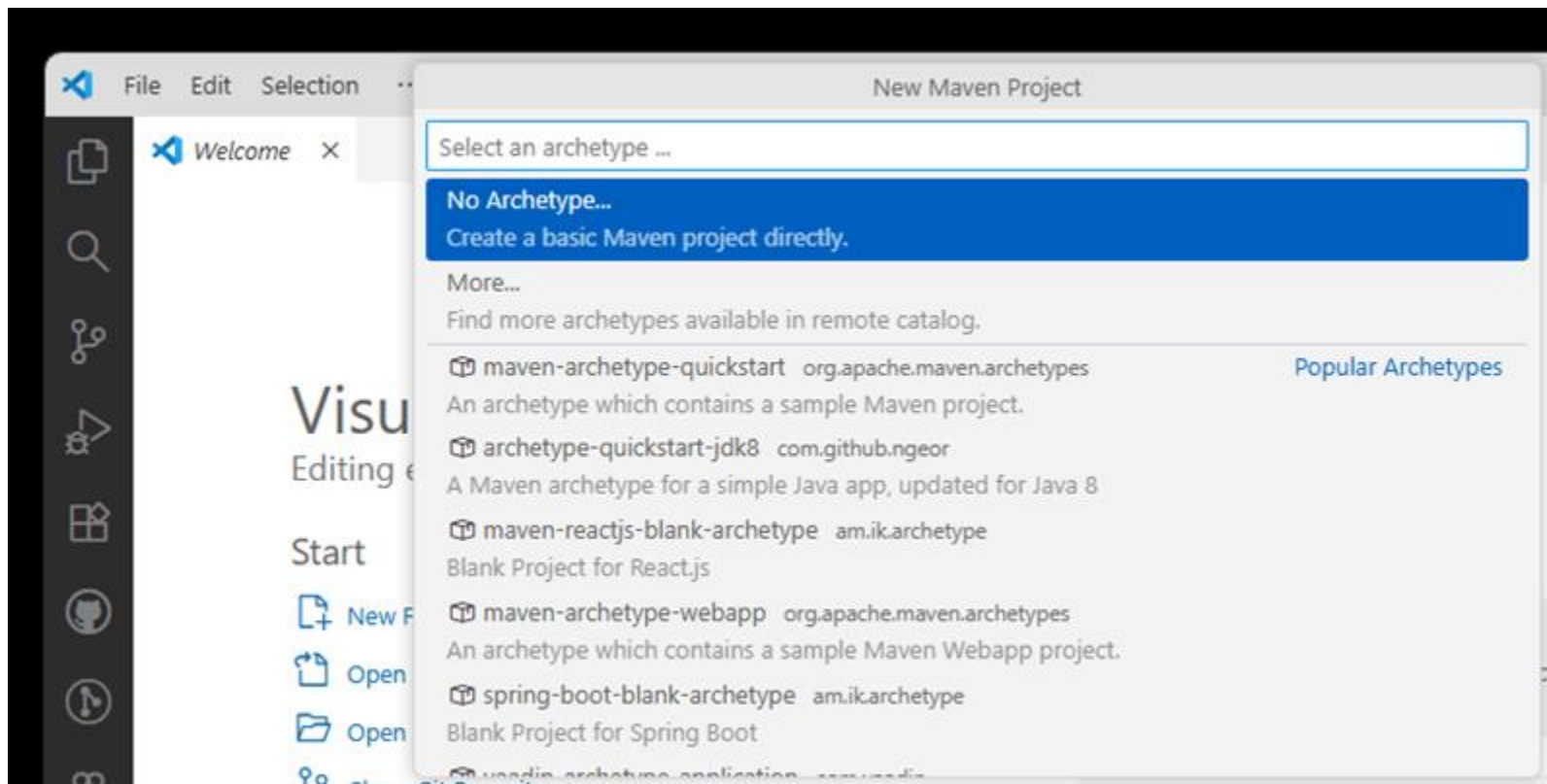
- **Spring Framework (com Spring Boot):** O padrão de mercado atual. Robusto, completo, mas com uma curva de aprendizado maior.
 - i. <https://spring.io/projects/spring-framework>
- **Javalin, Quarkus, Micronaut:** Foco em leveza, rapidez (inclusive no *startup*), e produtividade para APIs e microserviços.
 - i. <https://javalin.io/>
- ii. **Por que Javalin?**
 1. Extremamente leve e simples.
 2. "Não-mágico": O que você escreve é o que acontece.
 3. Inicia em milissegundos.
 4. Não requer um servidor de aplicação externo (ele embute o seu próprio, o Jetty).
 5. Perfeito para aprender os conceitos de HTTP e REST rapidamente.



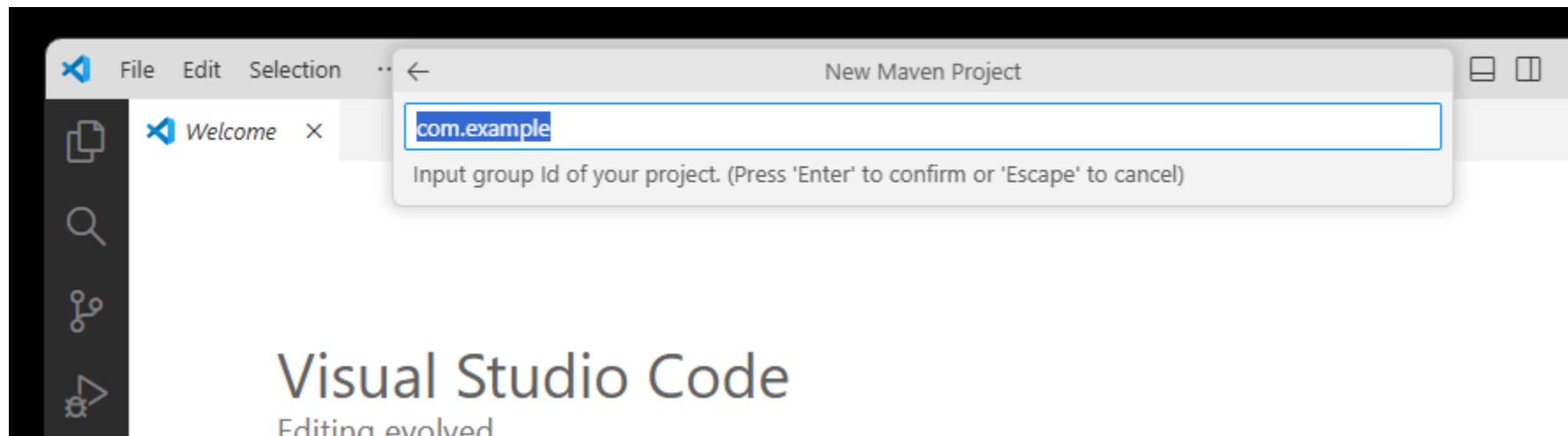
Criando um projeto



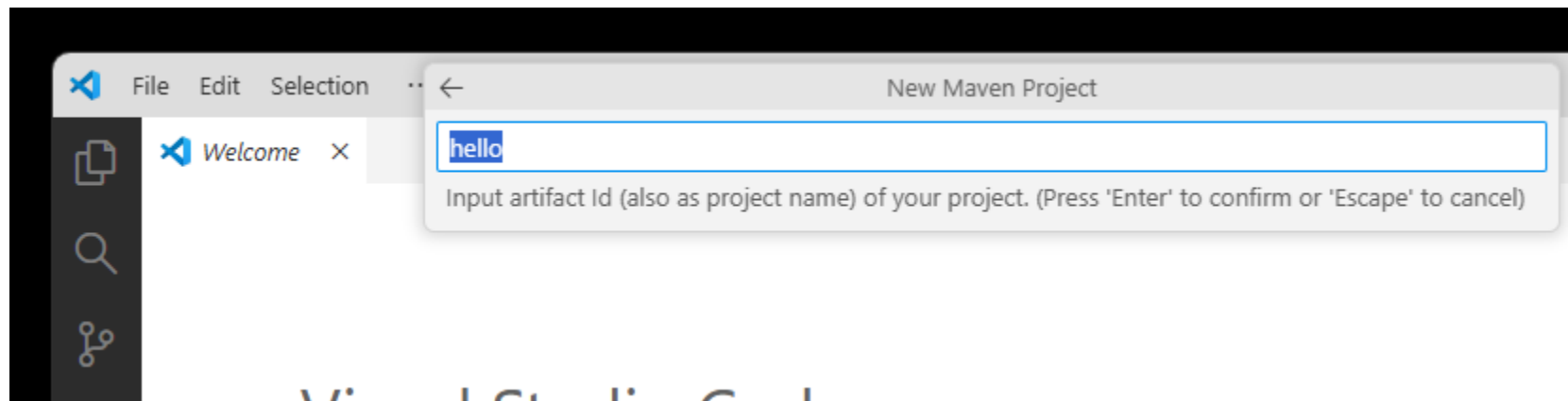
Criando um projeto



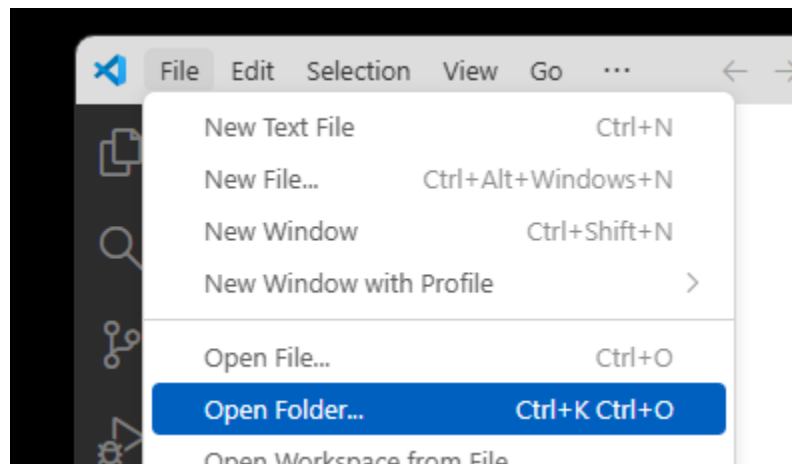
Criando um projeto



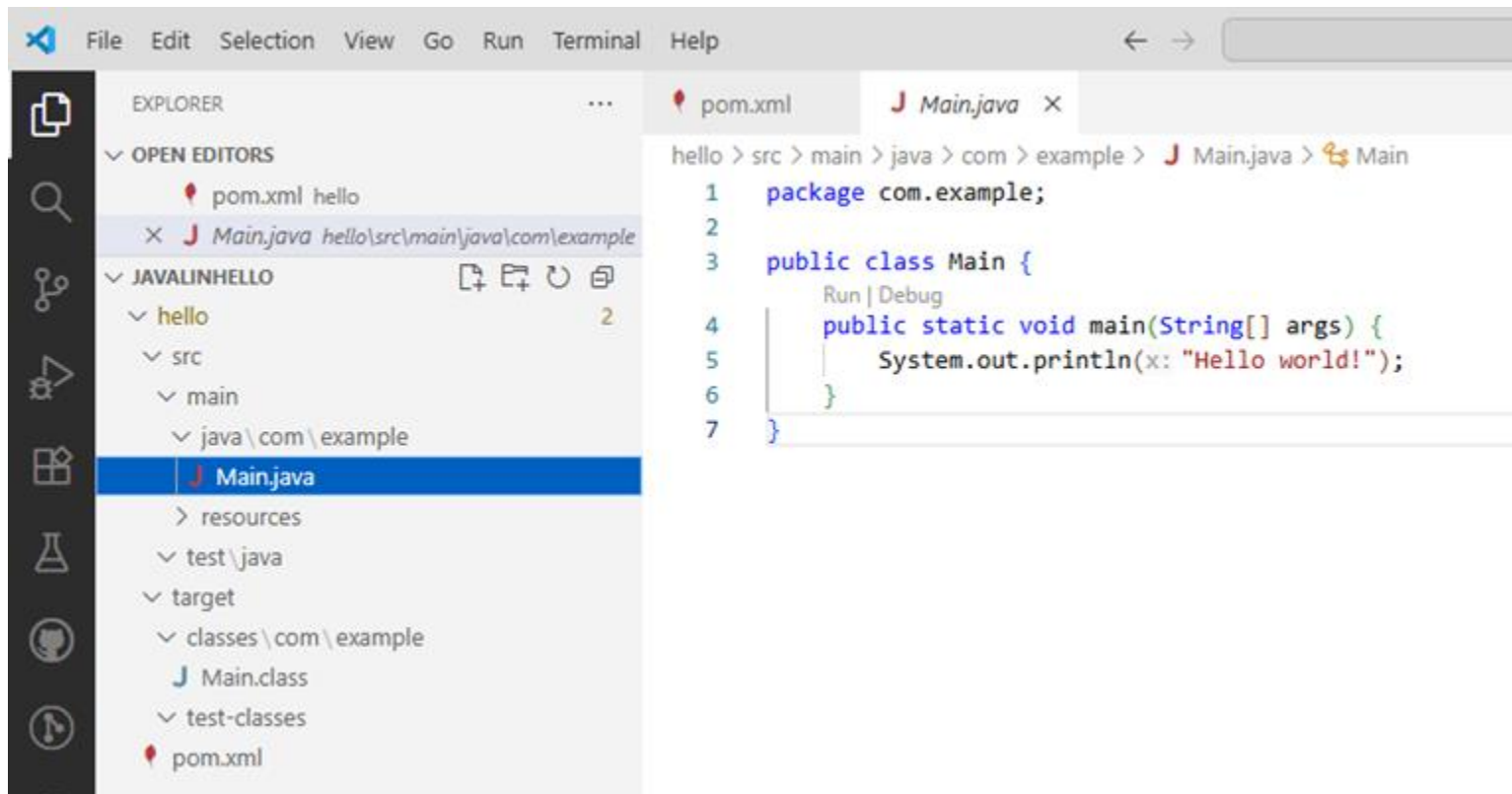
Criando um projeto



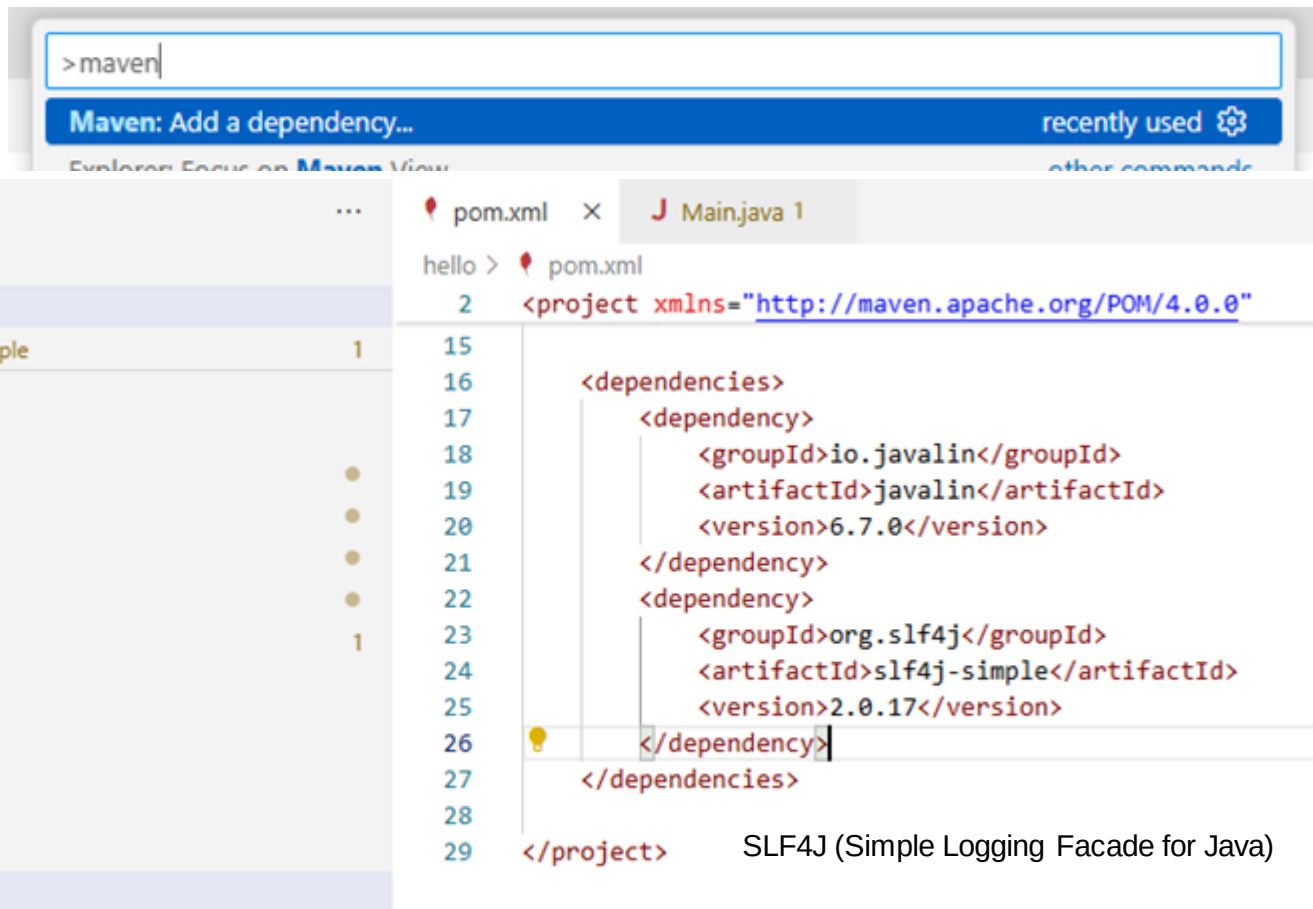
Criando um projeto



Criando um projeto



Criando um projeto: adicionando dependências



The screenshot displays the Visual Studio Code interface for a Maven project. At the top, a search bar contains the text ">maven". Below it, a dropdown menu shows the command "Maven: Add a dependency..." with a "recently used" button. The Explorer on the left shows the project structure, with "pom.xml" selected. The Editor on the right shows the "pom.xml" file with the following XML code:

```
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
15
16 <dependencies>
17   <dependency>
18     <groupId>io.javalin</groupId>
19     <artifactId>javalin</artifactId>
20     <version>6.7.0</version>
21   </dependency>
22   <dependency>
23     <groupId>org.slf4j</groupId>
24     <artifactId>slf4j-simple</artifactId>
25     <version>2.0.17</version>
26   </dependency>
27 </dependencies>
28
29 </project>
```

SLF4J (Simple Logging Facade for Java)

Criando um projeto

The screenshot shows an IDE interface with two main panels. The left panel, titled 'EXPLORER', displays a project tree. The tree is expanded to show the 'src/main/java/com/example' directory, where 'Main.java' is selected. The right panel shows the content of 'Main.java', which is a Java class using Javalin to create a simple web application that returns 'Hello World!'.

EXPLORER

- OPEN EDITORS
 - pom.xml hello
 - J Main.java** hello\src\main\java\com\example 1
- JAVALINHELLO
 - hello
 - src
 - main
 - java\com\example
 - J Main.java** 1
 - resources
 - test\java
 - target
 - classes\com\example
 - J Main.class
 - test-classes
 - pom.xml

Main.java

```
hello > src > main > java > com > example > J Main.java > Main
1  package com.example;
2
3  import io.javalin.Javalin;
4
5  public class Main {
6      Run | Debug
7      public static void main(String[] args) {
8          var app = Javalin.create()
9              .get("/", ctx -> ctx.result("Hello World!"))
10             .start(7070);
11      }
```

Construção do primeiro GET: "Hello World"

Construção do primeiro GET: "Hello World"

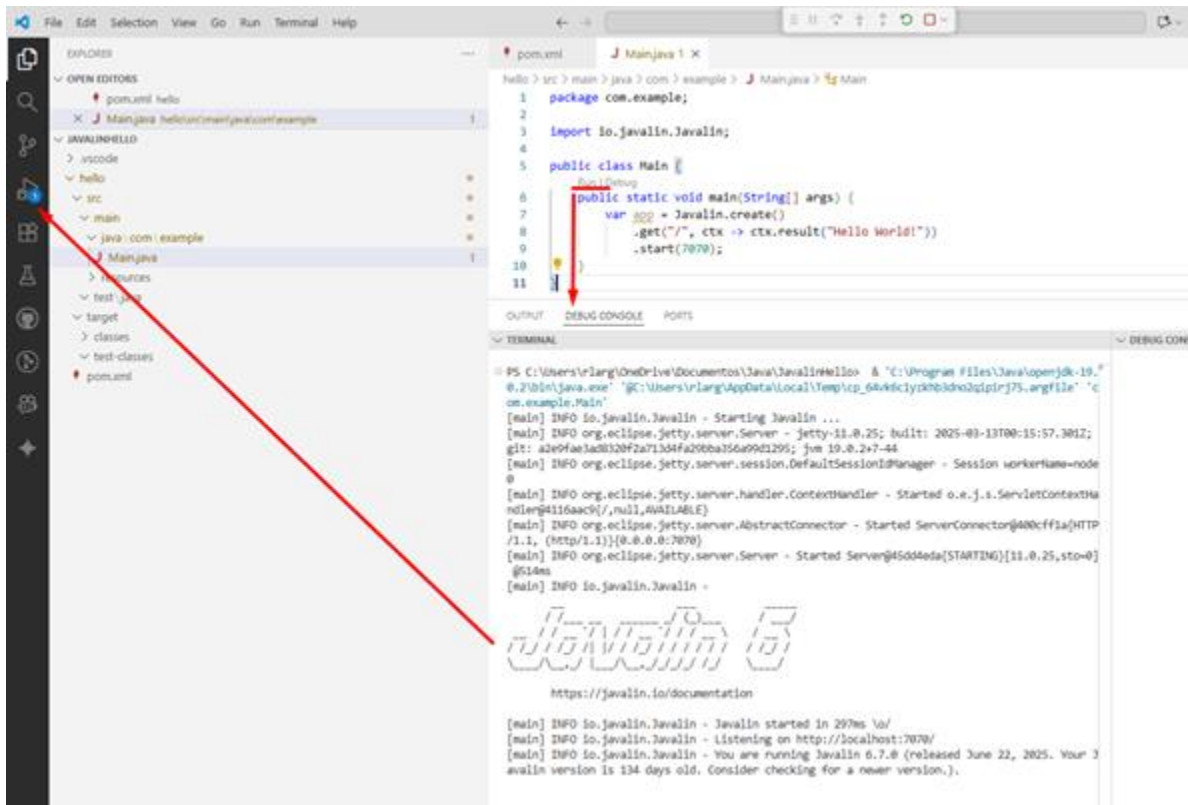
- **Conceito:** Criar um *endpoint* (manipulador) para a rota raiz (/) que responde a requisições HTTP GET.

Código (dentro do **main**):

```
app.get("/", ctx -> {  
    ctx.result("Hello World, Javalin!");  
});
```

- **Explicação:**
 - `app.get(...)`: Registra um *handler* (manipulador) para o verbo HTTP GET.
 - `" / "`: É o *path* (caminho) da rota.
 - `ctx -> { ... }`: É uma expressão Lambda. `ctx` é o objeto `Context`. Ele contém tudo sobre a requisição (request) e a resposta (response).
 - `ctx.result("...")`: Define o corpo (body) da resposta como um texto simples.
- **Teste:** Rodar a classe `App` e acessar `http://localhost:7070` no navegador.

Criando um projeto



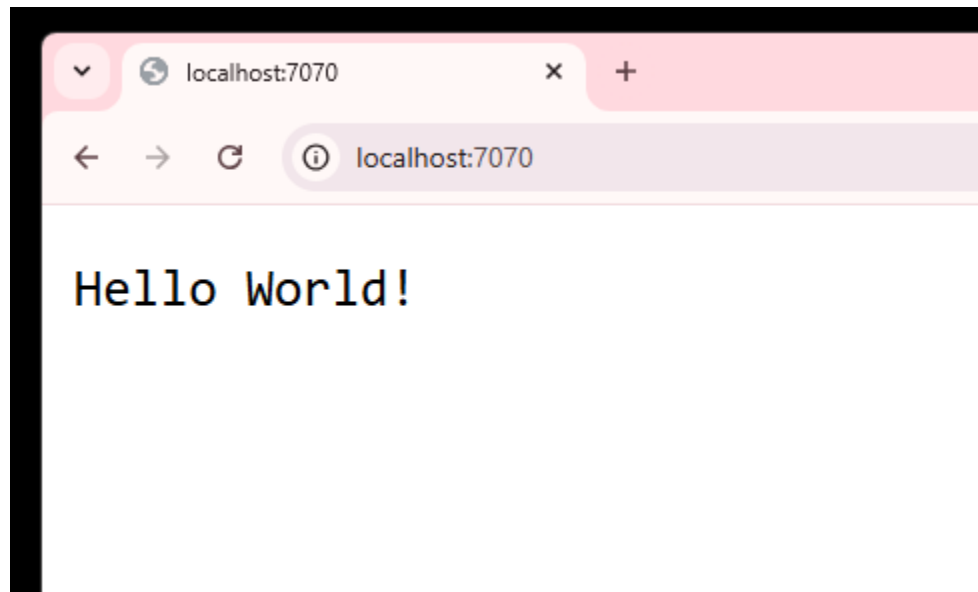
The screenshot shows an IDE with a project named 'JavalinHello'. The Explorer on the left shows the project structure, including 'pom.xml' and 'src/main/java/com/example/Main.java'. The Editor on the right shows the 'pom.xml' file with the following content:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
<groupId>com.example</groupId>
<artifactId>JavalinHello</artifactId>
<version>1.0.0</version>
<packaging>jar</packaging>
<name>JavalinHello</name>
<description>A Javalin Hello World</description>
<url>https://maven.apache.org</url>
<properties>
<java.version>11</java.version>
</properties>
<dependencies>
<dependency>
<groupId>com.example</groupId>
<artifactId>Javalin</artifactId>
<version>6.7.0</version>
</dependency>
</dependencies>
</project>
```

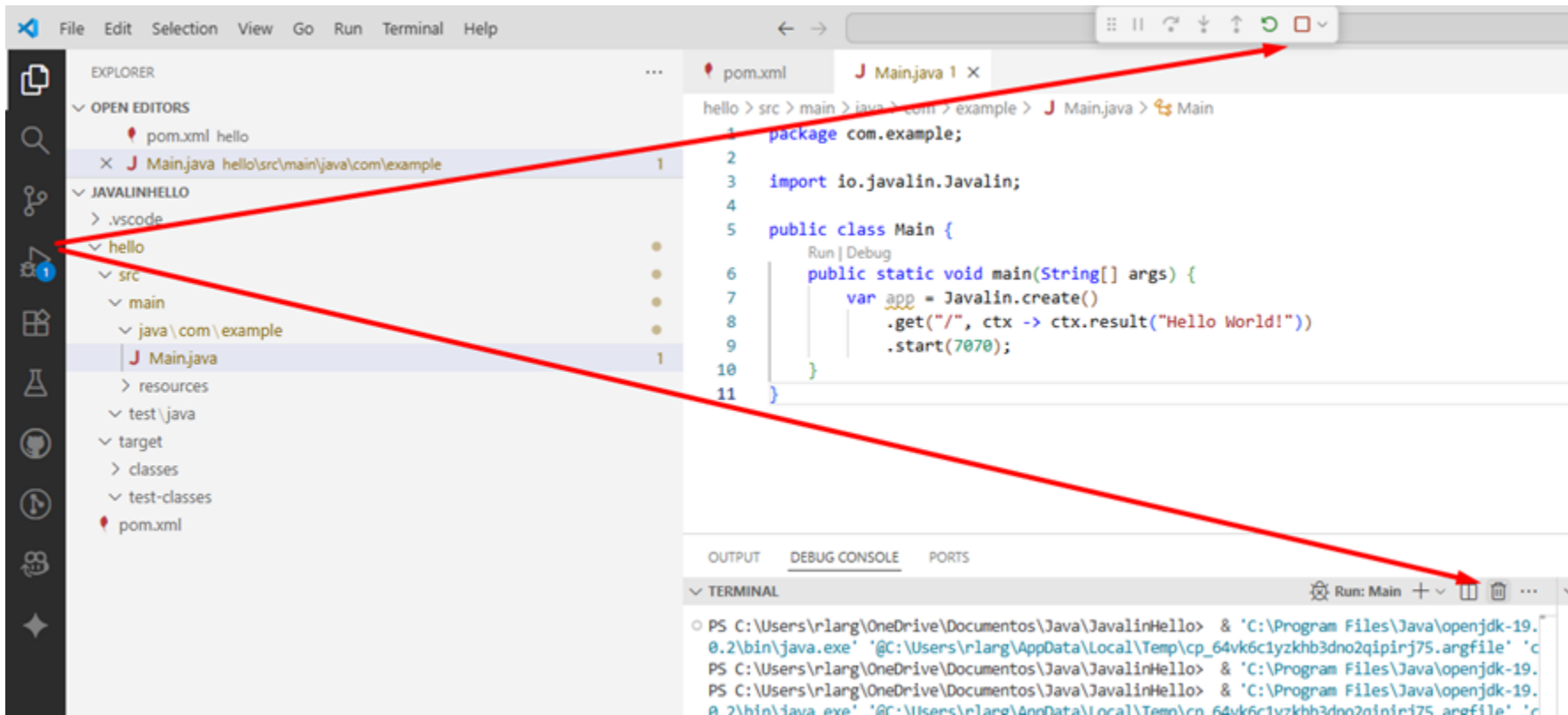
The Terminal at the bottom shows the output of running the application:

```
PS C:\Users\rlang\OneDrive\Documents\Java\JavalinHello> java -cp ".\target\classes;.\" -jar JavalinHello.jar
[main] INFO io.javalin.Javalin - Starting Javalin ...
[main] INFO org.eclipse.jetty.server.Server - jetty-11.0.25; built: 2025-09-13T00:15:57.301Z;
git: a2e9fae1ad320f2a71304fa20bba356a99d1295; jvm 19.0.2+7-44
[main] INFO org.eclipse.jetty.server.session.DefaultSessionIdManager - Session workerName=node
0
[main] INFO org.eclipse.jetty.server.handler.ContextHandler - Started o.e.j.s.ServletContextHa
ndler@4116aac9[/,null,AVAILABLE]
[main] INFO org.eclipse.jetty.server.AbstractConnector - Started ServerConnector@400cfff1a[HTTP
/1.1, (http/1.1)]{0.0.0.0:7070}
[main] INFO org.eclipse.jetty.server.Server - Started Server@45dd40da[STARTING]{11.0.25,sto=0}
@61d8a
[main] INFO io.javalin.Javalin -
Javalin
https://javalin.io/documentation
[main] INFO io.javalin.Javalin - Javalin started in 297ms \o/
[main] INFO io.javalin.Javalin - Listening on http://localhost:7070/
[main] INFO io.javalin.Javalin - You are running Javalin 6.7.0 (released June 22, 2025). Your J
avalin version is 134 days old. Consider checking for a newer version.)
```

Criando um projeto



Para parar o serviço



Criando um projeto

Use o GEMINI CLI ou outro codificador agêntico para te ajudar

- Mas **CUIDADO** com as versões

Passo 1: Adicionar as dependências do Javalin

Para usar o Javalin, primeiro precisamos informar ao Maven onde encontrá-lo. Fazemos isso adicionando "dependências" ao seu arquivo `pom.xml`. Pense nas dependências como bibliotecas ou ferramentas externas que seu projeto precisa para funcionar.

Além do Javalin, vamos adicionar o SLF4J, que é uma biblioteca de logging que o Javalin utiliza para mostrar mensagens informativas no console.

Vou aplicar as seguintes alterações ao seu `pom.xml`:

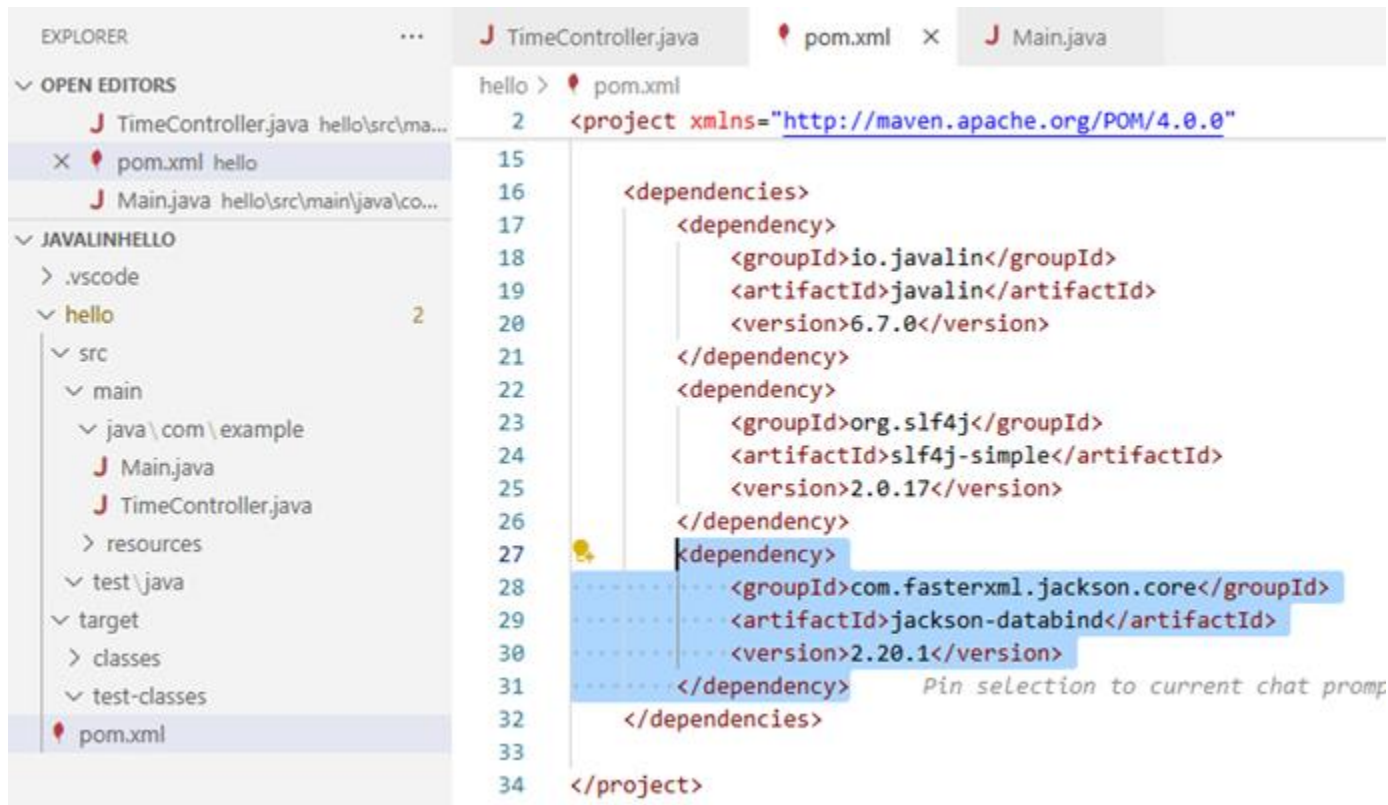
```
 pom.xml -0 +13
<?xml version="1.0" encoding="UTF-8" ?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>hello-world</artifactId>
  <version>1.0.0</version>
  <packaging>jar</packaging>
  <properties>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>
  <dependencies>
    <dependency>
      <groupId>io.javalin</groupId>
      <artifactId>javalin</artifactId>
      <version>6.1.3</version>
    </dependency>
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-simple</artifactId>
      <version>2.0.13</version>
    </dependency>
  </dependencies>
</project>
```

The screenshot shows an IDE with a Gemini chat window on the left and a Java code editor on the right. The chat window displays a conversation with 'Ricardo Pupo Larguesa' about creating a web project with Javalin. It includes a 'Thinking' status and a response explaining the steps to add dependencies to a Maven project. Below the chat, there's a 'Passo 1: Adicionar as dependências do Javalin' section with instructions and a diff view of the `pom.xml` file. The code editor on the right shows the `Main.java` file with the following code:

```
1 package com.example;
2
3 import io.javalin.Javalin;
4
5 public class Main {
6     public static void main(String[] args) {
7         System.out.println("Hello world!");
8         var app = Javalin.create()
9             .get("/", ctx -> ctx.result("Hello World!"))
10             .start(7070);
11     }
12 }
```

At the bottom of the IDE, there's an 'OUTPUT' tab showing the message 'Activating task providers java'.

Construção de GET JSON: Data/Hora do Servidor



The screenshot shows an IDE with the following components:

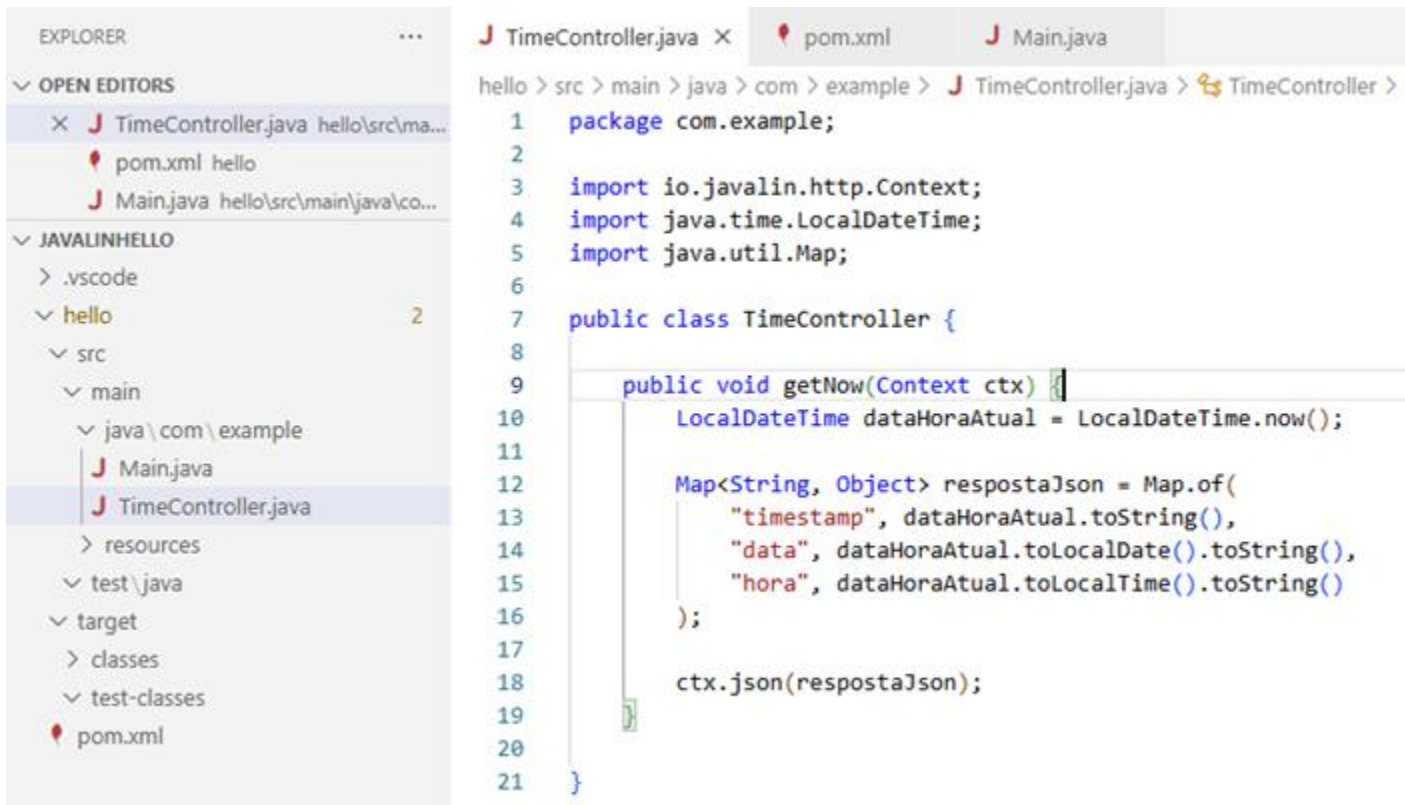
- EXPLORER:** Displays the project structure for 'JAVALINHELLO'. The 'pom.xml' file is selected under the 'hello' directory.
- OPEN EDITORS:** Lists the open files: 'TimeController.java', 'pom.xml', and 'Main.java'.
- Code Editor:** Displays the content of 'pom.xml'.

The 'pom.xml' file content is as follows:

```
hello > pom.xml
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
15
16 <dependencies>
17   <dependency>
18     <groupId>io.javalin</groupId>
19     <artifactId>javalin</artifactId>
20     <version>6.7.0</version>
21   </dependency>
22   <dependency>
23     <groupId>org.slf4j</groupId>
24     <artifactId>slf4j-simple</artifactId>
25     <version>2.0.17</version>
26   </dependency>
27   <dependency>
28     <groupId>com.fasterxml.jackson.core</groupId>
29     <artifactId>jackson-databind</artifactId>
30     <version>2.20.1</version>
31   </dependency>
32 </dependencies>
33
34 </project>
```

A blue selection box highlights the Jackson dependency block (lines 27-31). A tooltip is visible over the selection, displaying the text: "Pin selection to current chat prompt".

Construção de GET JSON: Data/Hora do Servidor



The image shows a Visual Studio Code editor with a project named 'hello'. The Explorer sidebar on the left shows the project structure, with the file 'TimeController.java' selected under the path 'src/main/java/com/example'. The main editor area displays the code for 'TimeController.java'.

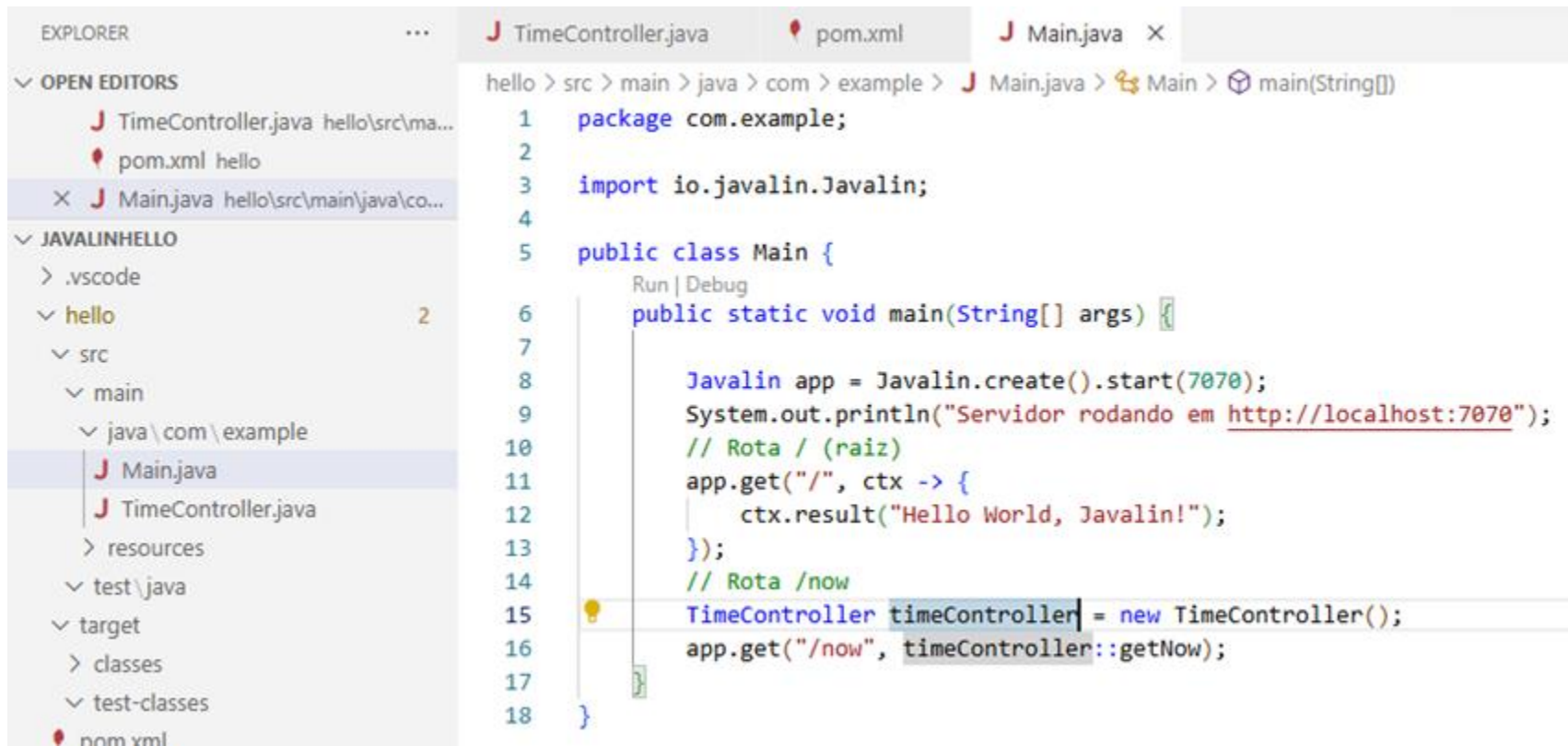
Project Structure (Explorer):

- hello
 - .vscode
 - hello
 - src
 - main
 - java\com\example
 - Main.java
 - TimeController.java
 - resources
 - test\java
 - target
 - classes
 - test-classes
 - pom.xml

Code in TimeController.java:

```
1 package com.example;
2
3 import io.javalin.http.Context;
4 import java.time.LocalDateTime;
5 import java.util.Map;
6
7 public class TimeController {
8
9     public void getNow(Context ctx) {
10         LocalDateTime dataHoraAtual = LocalDateTime.now();
11
12         Map<String, Object> respostaJson = Map.of(
13             "timestamp", dataHoraAtual.toString(),
14             "data", dataHoraAtual.toLocalDate().toString(),
15             "hora", dataHoraAtual.toLocalTime().toString()
16         );
17
18         ctx.json(respostaJson);
19     }
20
21 }
```

Construção de GET JSON: Data/Hora do Servidor



The image shows a Visual Studio Code editor window with a project named 'hello'. The Explorer sidebar on the left displays the file structure:

- EXPLORED
- OPEN EDITORS
 - TimeController.java hello\src\ma...
 - pom.xml hello
 - Main.java hello\src\main\java\co...
- JAVALINHELLO
 - .vscode
 - hello
 - src
 - main
 - java\com\example
 - Main.java
 - TimeController.java
 - resources
 - test\java
 - target
 - classes
 - test-classes
 - nom.xml

The main editor area shows the code for `Main.java` in the package `com.example`. The code is as follows:

```
1 package com.example;
2
3 import io.javalin.Javalin;
4
5 public class Main {
6     Run | Debug
7     public static void main(String[] args) {
8         Javalin app = Javalin.create().start(7070);
9         System.out.println("Servidor rodando em http://localhost:7070");
10        // Rota / (raiz)
11        app.get("/", ctx -> {
12            ctx.result("Hello World, Javalin!");
13        });
14        // Rota /now
15        TimeController timeController = new TimeController();
16        app.get("/now", timeController::getNow);
17    }
18 }
```

Construção de GET JSON: Data/Hora do Servidor

← → ↻ ⓘ localhost:7070/now

Estilos de formatação 

```
{  
  "timestamp": "2025-11-04T08:43:00.185458700",  
  "data": "2025-11-04",  
  "hora": "08:43:00.185458700"  
}
```

Construção de GET JSON: Data/Hora do Servidor

Instanciação: Criamos o objeto `TimeController timeController = new TimeController();`. Isso é Programação Orientada a Objetos: não podemos chamar um método sem ter um objeto (a menos que o método seja `static`, o que evitaríamos aqui).
`timeController::getNow`: Esta sintaxe estranha chama-se **Method Reference** (Referência de Método).

- É um atalho simples do Java para uma lambda.
- Escrever `timeController::getNow` é **exatamente a mesma coisa** que escrever `ctx -> timeController.getNow(ctx)`.
- Estamos dizendo ao Javalin: "Quando uma requisição GET chegar em `/now`, por favor, execute o método `getNow` do *objeto* `timeController` e passe o `Context` para ele."
- `ctx.json(objeto)`: Serializa qualquer objeto Java (POJO, Map, List) para uma string JSON, e define o `Content-Type` da resposta como `application/json`.



TAREFA: Greeting por Horário

Objetivo: Implementar o endpoint `GET /greeting`.

Requisitos:

1. Deve responder com **texto plano** (`ctx.result()`).
2. O texto deve conter uma saudação.
3. A saudação muda com base na hora do dia:
 - 05:00 - 11:59: "Bom dia"
 - 12:00 - 17:59: "Boa tarde"
 - 18:00 - 23:59: "Boa noite"
 - 00:00 - 04:59: "Vá dormir"

Dica: Usar `LocalDateTime.now().getHour()` e uma estrutura `if/else if/else`.

EXEMPLO: Números aleatórios

Implementar um endpoint `GET '/random/{n}'`, que receba um número `n` como **parâmetro de caminho** (path parameter). O endpoint deve retornar um JSON contendo a quantidade solicitada (`n`) e um *array* com `n` números aleatórios (ex: de 0 a 100).


```
EXPLORER
...
JAVAHELLOJAVALIN
> .vscode
> hello
  > src
    > main
      > java\com\example
        > Main.java
        > NumbersController.java
        > TimeController.java
      > resources
      > test\java
      > target
      > pom.xml

NumbersController.java
hello > src > main > java > com > example > NumbersController.java > getRandomNumbers(Context)
1 package com.example;
2
3 import java.util.ArrayList;
4 import java.util.List;
5 import java.util.Map;
6 import java.util.Random;
7
8 import io.javalin.http.Context;
9
10 public class NumbersController {
11
12     public void getRandomNumbers(Context ctx) {
13         try {
14             int n = Integer.parseInt(ctx.pathParam("n"));
15
16             if (n < 0) {
17                 ctx.status(400).json(Map.of(
18                     "error",
19                     "A quantidade deve ser um número não-negativo."
20                 ));
21                 return;
22             }
23
24             Random random = new Random();
25             List<Integer> numbers = new ArrayList<>();
26             for (int i = 0; i < n; i++) {
27                 numbers.add(random.nextInt(101)); // Gera um número entre 0 e 100
28             }
29
30             ctx.json(Map.of("quantity", n, "numbers", numbers));
31
32         } catch (NumberFormatException e) {
33             ctx.status(400).json(Map.of("error", "O parâmetro deve ser um inteiro válido.));
34         }
35     }
36 }
```

```
Main.java
hello > src > main > java > com > example > Main.java > Main > main(String[])
1 package com.example;
2
3 import io.javalin.Javalin;
4
5 public class Main {
6     public static void main(String[] args) {
7         Javalin app = Javalin.create().start(7070);
8
9         app.get("/", ctx -> ctx.result("Hello World, Javalin!"));
10
11         TimeController timeController = new TimeController();
12         app.get("/now", timeController::getNow);
13         app.get("/greeting", timeController::getGreeting);
14
15         NumbersController numbersController = new NumbersController();
16         app.get("/random/{n}", numbersController::getRandomNumbers);
17     }
18 }
```

localhost:7070/random/20

localhost:7070/random/20

Estilos de formatação

```
{
  "quantity": 20,
  "numbers": [48, 8, 75, 62, 46, 22, 98, 78, 34, 19, 86, 73, 9, 17, 75, 37, 74, 75, 23, 44]
}
```

EXEMPLO: Tabuada

Implementar um endpoint `GET '/tabuada'`, que receba um número `n` como **parâmetro de caminho** (path parameter). O endpoint deve retornar um JSON contendo o número `n` e um *array* de strings com a tabuada completa desse número (do 1 ao 10).

```

NumbersController.java
hello > src > main > java > com > example > J NumbersController.java > NumbersController > getTabuada(Context)
5 import java.util.ArrayList;
6 import java.util.List;
7 import java.util.Map;
8 import java.util.Random;
9
10 public class NumbersController {
11
12     public void getRandomNumbers(Context ctx) { ...
13
14     public void getTabuada(Context ctx) {
15         try {
16             int n = Integer.parseInt(ctx.pathParam("n"));
17
18             List<String> tabuada = new ArrayList<>();
19             for (int i = 1; i <= 10; i++) {
20                 tabuada.add(n + " x " + i + " = " + (n * i));
21             }
22
23             ctx.json(Map.of(
24                 "number", n,
25                 "tabuada", tabuada
26             ));
27
28         } catch (NumberFormatException e) {
29             ctx.status(400).json(Map.of(
30                 "error", "O parâmetro deve ser um inteiro válido. " + e.getMessage()
31             ));
32         }
33     }
34 }

```

```

Main.java
hello > src > main > java > com > example > J Main.java > Main
1 package com.example;
2
3 import io.javalin.Javalin;
4
5 public class Main {
6     Run | Debug
7     public static void main(String[] args) {
8         Javalin app = Javalin.create().start(7070);
9
10         app.get("/", ctx -> ctx.result("Hello World, Javalin!"));
11
12         TimeController timeController = new TimeController();
13         app.get("/now", timeController::getNow);
14         app.get("/greeting", timeController::getGreeting);
15
16         NumbersController numbersController = new NumbersController();
17         app.get("/random/{n}", numbersController::getRandomNumbers);
18         app.get("/tabuada/{n}", numbersController::getTabuada);
19     }
20 }

```

localhost:7070/tabuada/8

localhost:7070/tabuada/8

Estilos de formatação ☒

```

{
  "number": 8,
  "tabuada": [
    "8 x 1 = 8",
    "8 x 2 = 16",
    "8 x 3 = 24",
    "8 x 4 = 32",
    "8 x 5 = 40",
    "8 x 6 = 48",
    "8 x 7 = 56",
    "8 x 8 = 64",
    "8 x 9 = 72",
    "8 x 10 = 80"
  ]
}

```

EXEMPLO: Soma

Implementar um endpoint `GET '/soma'`, que receba dois números, `n1` e `n2`, como **parâmetros de consulta** (query parameters, ex: `/soma?n1=10.5&n2=20`). O endpoint deve retornar um JSON com os valores de `n1`, `n2` e o `resultado` da soma entre eles.

```

NumbersController.java X
hello > src > main > java > com > example > J NumbersController.java > NumbersController > getSoma(Context)
5 import java.util.ArrayList;
6 import java.util.List;
7 import java.util.Map;
8 import java.util.Random;
9
10 public class NumbersController {
11
12 > public void getRandomNumbers(Context ctx) { ...
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27 > public void getTabuada(Context ctx) { ...
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48 public void getSoma(Context ctx) {
49     try {
50         double n1 = Double.parseDouble(ctx.queryParam("n1"));
51         double n2 = Double.parseDouble(ctx.queryParam("n2"));
52
53         double sum = n1 + n2;
54
55         ctx.json(Map.of(
56             "n1", n1,
57             "n2", n2,
58             "sum", sum
59         ));
60     } catch (NumberFormatException | NullPointerException e) {
61         ctx.status(400).json(Map.of(
62             "error", "Os parâmetros de consulta 'n1' e 'n2' são obrigatórios e devem ser números válidos."
63         ));
64     }
65 }
66
67
68
69
70
71
72
73
74
75
76
77 }

```

```

Main.java X
hello > src > main > java > com > example > J Main.java > Main
1 package com.example;
2
3 import io.javalin.Javalin;
4
5 public class Main {
6     Run | Debug
7     public static void main(String[] args) {
8         Javalin app = Javalin.create().start(7070);
9
10         app.get("/", ctx -> ctx.result("Hello World, Javalin!"));
11
12         TimeController timeController = new TimeController();
13         app.get("/now", timeController::getNow);
14         app.get("/greeting", timeController::getGreeting);
15
16         NumbersController numbersController = new NumbersController();
17         app.get("/random/{n}", numbersController::getRandomNumbers);
18         app.get("/tabuada/{n}", numbersController::getTabuada);
19         app.get("/soma", numbersController::getSoma);
20     }
21 }

```

localhost:7070/soma?n1=2&n2=3

Estilos de formatação ☒

```

{
  "n1": 2,
  "n2": 3,
  "sum": 5
}

```



TAREFA: Loteria (JSON com Array)

Objetivo: Implementar o endpoint `GET /loteria`.

Requisitos:

1. Criar uma nova classe chamada `LoteriaController`.
2. Deve responder com **JSON**.
3. O JSON deve conter:
 - `timestamp`: A data/hora atual.
 - `numeros`: Um *array* com 6 números aleatórios.
4. **Desafio:** Os 6 números devem ser **únicos** (sem repetição), entre 1 e 60.

Dica: Usem um `Set<Integer>` para garantir a unicidade. Continuem adicionando números aleatórios (de 1 a 60) ao `Set` dentro de um loop `while`, até que o `set.size()` seja 6.